Python 3

Fernando Anselmo - Versão 1.0

Variáveis e E/S

Comentários:

```
# Linha comentada
```

Variáveis e Tipos de Dados:

Classe String: `nome = 'Alice' ou "Alice"`
Também permite 3 aspas simples ou duplas

Classe Integer: `idade = 30`
`0b010` (binário) `0o642` (octal) `0xF3` (hexadecimal)

Classe Float: `altura = 1.50`
Notação científica: `altura = 1.7e-6`

Classe Boolean: `e_curiosa = True`

Classe NoneType: `nascimento = None`

Inicialização Conjunta:

```
a = b = c = 20
a, b, c = 20, 21, 22
```

Entrada e Saída de Informação:

```
nome = input("Informe seu nome:  ")
valor = eval(input("Informe um valor:"))
print(nome)
print(f"Possuo valor")
```

Casting:

```
str(), int(), float(), bool()
```

Operadores

Aritméticos:

`+` (adição)
`-` (subtração)
`*` (multiplicação)
`/` (divisão real)
`//` (divisão inteiro)
`%` (resto da divisão)
`**` (exponenciação)

Comparação:

`==`, `!=`, `>`, `<`, `>=` e `<=`

Concatenadores Lógicos:

`and`, `or` e `not`

Identificadores:

`is`, `not` e `in`

Controle de Fluxo

Comando de Decição if / elif / else:

```
idade = eval(input("Qual sua idade?"))
if idade < 18:
  print("Você é de menor.")
elif idade < 60:
  print("Você é adulto.")
else:
  print("Você é sênior.")
```

Laço for (interage somente com as Estruturas de Dados e a Classe String):

```
for item in ['maçã', 'banana']:
  print(item)
else:
  print("Terminado")

for i in range(5):
  print(i)
```

Laço while (gera um laço condicional):

```
vezes = eval(input("Quantas vezes deseja
repetir?  "))
while vezes > 0:
  print(vezes, 'vez')
  vezes -= 1
else:
  print("Terminado")
```

Bloco de Proteção

São blocos utilizados para proteger o programa de **Entrada Externas** errôneas, que escapam do controle do programa:

```
valor = input("Informe um valor:  ")
try:
  # tenta converter para valor numérico
  valor = eval(valor)
except ValueError:
  print("Entrada Inválida")
else:
  print("Nenhum erro aconteceu!")
finally:
  print("Sempre vai executar!")
```

Estrutura de Dados

Listas - Classe List ▷ a = [] (mutáveis):

```
minha_lst = [1, "a", 2, "b"]
print(minha_lst[1]) # Mostra a
minha_lst.append(v) # Adicionar final
minha_lst.insert(p,v) # Adic.  na pos.  'p'
minha_lst.remove(v) # Remover o elemento
minha_lst.pop() # Remover do final
print(len(minha_lst)) # Tamanho da Lista
minha_lst.sort() # Ordernar os valores
minha_lst.reverse() # Inverter a lista
```

Tuplas - Classe Tuple ▷ a = () (imutáveis):

```
minha_tup = (1, "a", 2, "b")
minha_tup[1] = 2 # produz um erro
```

Conjuntos - Classe Set ▷ a = {0} (elementos únicos):

```
meu_set = {1, 2, 2, 2, 3, 3, 3}
print(len(meu_set)) # Mostra 3
```

Dicionários - Classe Dict ▷ a = {} (chaves únicas):

```
meu_dict = {"nome":  "Alice", "idade":  30}
print(meu_dic["nome"]) # Mostra Alice
meu_dic["cidade"] = 'País das Maravilhas'
meu_dic.keys() # Obtém as chaves
meu_dic.values() # Obtém os valores
meu_dic.items() # Obtém chaves e valores
```

Funções e List Comprehensions

Funções:

```
def saudacao(nome="World"):
  return f"Hello, {nome}"

print(saudacao("Python"))
print(saudacao())
```

List Comprehensions:
Este é um meio rápido para se obter uma lista, criar uma lista de valores ao quadrado de 0 a 4

```
quadrados = [i ** 2 for i in range(5)]
```

Página do Django: `https://www.python.org`